Лабораторная работа №3. Что лежит в репозитории: секреты и зависимости

Время: 4 академических часа (два занятия): часть А — секреты и Gitleaks, часть Б — зависимости, Trivy и SBOM.

Что получится: вы найдёте секрет в файлах и в истории Git, поставите блокировку коммитов с секретами, оформите исключение; затем просканируете библиотеки проекта, сформируете SBOM, отделите опасные находки от неприменимых и настроите порог для конвейера.

Код писать не нужно: исправления подготовлены.

Перед началом: выполнена ЛР2, установлен и запущен Docker.

---

1. Цель работы

В ЛР2 сканер проверял код, который написали разработчики проекта. Но в репозитории есть ещё две вещи, за которые отвечает команда: **секреты**, попавшие в файлы по ошибке, и **чужой код** — библиотеки. Цель работы — научиться проверять и то и другое.

После работы вы сможете:

- объяснить, почему удаление секрета из файла не устраняет утечку;
- запускать Gitleaks по файлам и по истории, читать его находки;
- настроить блокировку коммитов через pre-commit;
- оформить исключение так, чтобы оно не стало способом прятать проблемы;
- запускать Trivy, читать таблицу уязвимостей, формировать SBOM;
- отличать «уязвимость найдена» от «уязвимость опасна для нас»;
- задать порог, при котором сканер останавливает конвейер.

2. Теория

2.1. Секреты

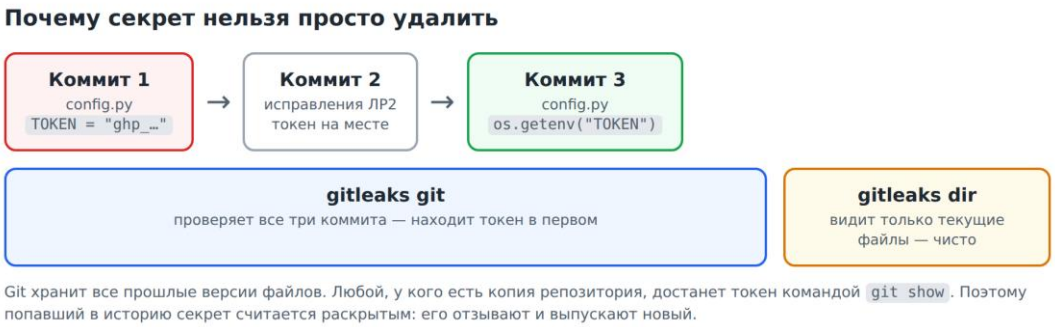
Секрет — строка, которая сама по себе даёт доступ: пароль, токен API, закрытый ключ, строка подключения к базе с паролем. Кто знает секрет, тот действует от имени его владельца.

Секреты попадают в код по понятным причинам: «временно вписал для проверки», «репозиторий же закрытый», «потом уберу». Автоматические программы круглосуточно просматривают публичные репозитории, и опубликованный токен находят за минуты.

В `backend/config.py` учебного проекта лежит строка, похожая на токен GitHub. Она **фиктивная** и доступа никуда не даёт. Работать с ней будем как с настоящей.

2.2. Почему секрет нельзя просто удалить

Git хранит не только текущее состояние файлов, но и все прошлые.



Если удалить токен и сделать новый коммит, старый коммит с токеном останется в истории. Любой, у кого есть копия репозитория, достанет его одной командой. Отсюда главное правило:

> Секрет, попавший в репозиторий, считается раскрытым. Его нужно **отозвать и выпустить новый**. Удаление из кода — только уборка после этого.

Порядок действий при настоящей утечке:

№	Действие	Зачем
1	**Отозвать** секрет у того, кто его выдал	украденная копия перестаёт работать
2	**Выпустить новый** и передать приложению безопасным способом	сервис продолжает работать
3	**Проверить журналы** — пользовался ли кто-то секретом	понять, был ли ущерб
4	**Убрать** секрет из кода	чтобы не повторить утечку при следующем копировании
5	**Устранить причину** — поставить автоматическую проверку	чтобы ситуация не повторилась

Переписывание истории Git технически возможно, но не заменяет отзыва: неизвестно, кто уже успел сделать копию.

2.3. Где хранить секреты

Код и секреты должны жить отдельно. Простейший способ — **переменные окружения**: значение задаётся там, где программу запускают, а в коде остаётся только имя переменной.

Плохо	Хорошо
`TOKEN = "ghp_8ae31c…"`	`TOKEN = os.getenv("TOKEN")`

В промышленных системах используют хранилища секретов, а в конвейерах CI — защищённые переменные (в GitHub они называются Secrets).

2.4. Как Gitleaks находит секреты

Два способа:

- **Правила для известных форматов.** У токенов популярных сервисов узнаваемый вид: токен GitHub начинается с `ghp_`, ключ AWS — с `AKIA`. Для сотен таких форматов есть готовые правила.
- **Энтропия.** Это мера случайности строки. У слова `password` энтропия низкая, у `8ae31cacf141669d` — высокая. Длинная случайная строка рядом со словами `token`, `key`, `secret` подозрительна, даже если формат неизвестен. Так работает правило `generic-api-key`.

Два режима:

Команда	Что проверяет	Когда нужна
`gitleaks dir`	файлы в их текущем виде	быстрая проверка рабочей копии
`gitleaks git`	все коммиты в истории	полная проверка репозитория, конвейер

Каждая находка получает **отпечаток (fingerprint)** — строку вида `коммит:файл:правило:строка`, которая однозначно её определяет. По отпечатку находку можно внести в исключения.

Ограничение: слабый, но «неслучайный» секрет — например, пароль `admin123` или ключ `dev-only-change-me` — по энтропии не выделяется, и Gitleaks его не заметит.

2.5. pre-commit: проверка до того, как секрет ушёл

Git умеет запускать программы в определённые моменты — например, перед созданием коммита. Такие программы называют **хуками** (hooks). Если хук завершился с ошибкой, коммит не создаётся.

pre-commit — менеджер хуков: в файле `.pre-commit-config.yaml` перечислены проверки, команда `pre-commit install` подключает их к репозиторию.

Где проверка	Достоинство	Недостаток
pre-commit на компьютере разработчика	секрет не покидает компьютер	можно не установить или обойти (`--no-verify`)
CI на сервере	обойти нельзя; проверяется вся история	к этому моменту секрет уже отправлен

Нужны обе: первая — для скорости, вторая — для гарантии.

2.6. Чужой код в вашем проекте

Современное приложение на 80–90 % состоит из кода, который команда не писала.

Из чего состоит то, что вы запускаете

Слой	Пример в нашем проекте	Кто проверяет
Ваш код	backend/app.py, шаблоны	SAST — Semgrep
Прямые зависимости	Flask, psycopg, gunicorn — из requirements.txt	SCA — `trivy fs`
Транзитивные зависимости	Werkzeug, Jinja2, click — их привёл Flask	SCA — `trivy image`
Базовый образ	python:3.11-slim — это Debian с сотней пакетов	SCA — `trivy image`
Настройки запуска	Dockerfile, compose.yaml	`trivy config` и человек

Своего кода в образе — несколько процентов. Всё остальное написали другие люди, и уязвимости в этом коде находят и публикуют каждый день.

- **Прямая зависимость** — библиотека, которую вы сами указали в `requirements.txt`. У нас это Flask.
- **Транзитивная зависимость** — библиотека, которую привела за собой прямая. Flask приводит Werkzeug, Jinja2 и другие.

Уязвимость в любой из них — уязвимость вашего приложения, хотя вы не написали ни строки этого кода.

2.7. SCA, CVE и база уязвимостей

SCA (Software Composition Analysis) — анализ состава ПО. Сканер:

1. находит в проекте списки зависимостей;
2. определяет название и версию каждого компонента;
3. сверяет их с базами известных уязвимостей.

SCA не читает код библиотек и не ищет в них новые ошибки. Он отвечает на вопрос: «есть ли у нас версии, про которые **уже известно**, что они уязвимы?»

Термин	Значение
CVE	идентификатор известной уязвимости, например `CVE-2019-19844`
Severity	тяжесть: LOW, MEDIUM, HIGH, CRITICAL — словесная форма оценки CVSS (0–10)
Installed Version	версия, которая указана в проекте
Fixed Version	версия, в которой уязвимость исправлена
Status	`fixed` — исправление есть; `affected` — уязвимость есть, исправления пока нет

База уязвимостей пополняется каждый день. Поэтому проект, в котором никто ничего не менял, со временем «краснеет» сам: в его старых библиотеках находят новые дыры.

2.8. «Найдено» не значит «опасно»

Тяжесть CVE описывает уязвимость «в вакууме». Риск для конкретного проекта зависит от вопросов, на которые сканер не отвечает:

- используется ли компонент вообще?
- попадает ли он в то, что запускается?
- вызывается ли уязвимая функция и доступна ли она нарушителю?

Поэтому отчёт SCA — не приговор, а список вопросов для разбора. Варианты решения:

Решение	Когда
обновить до Fixed Version	компонент нужен, исправление есть
удалить компонент	компонент не используется
компенсирующая мера	исправления нет, но можно закрыть путь атаки иначе
принять риск на срок	уязвимая функция не используется; с обоснованием и датой пересмотра

2.9. SBOM

SBOM (Software Bill of Materials) — перечень компонентов программы: названия, версии, типы. Аналог состава на упаковке продукта.

Зачем он, если есть сканер? Представьте: завтра объявляют о критической уязвимости в популярной библиотеке. Вопрос руководства — «затронуты ли мы и в каких версиях продукта?». Без SBOM нужно поднять и пересканировать каждый выпуск. С SBOM — это поиск по файлам за минуту.

Распространённые форматы — **CycloneDX** и SPDX. SBOM сам уязвимости не находит: это опись, а не проверка.

---

3. Ход работы

```
cd ~/devsecops-lab
source .venv/Scripts/activate      # Windows, Git Bash
# source .venv/bin/activate        # Linux и macOS
mkdir -p reports
```

Только Windows (Git Bash) — выполняйте в каждом новом окне терминала перед командами `docker run`:

```
export MSYS_NO_PATHCONV=1
```

Без этого Git Bash превращает путь `/src` внутри контейнера в путь Windows, и сканер не находит файлы.

Часть A. Секреты

Шаг A1. Проверить, что сканер запускается

```
docker run --rm ghcr.io/gitleaks/gitleaks:v8.24.2 version
```

Что увидите: `8.24.2`. Первый запуск скачивает образ (около минуты).

С этой работы сканеры запускаются через Docker. Разберите команду один раз — дальше она везде устроена так же:

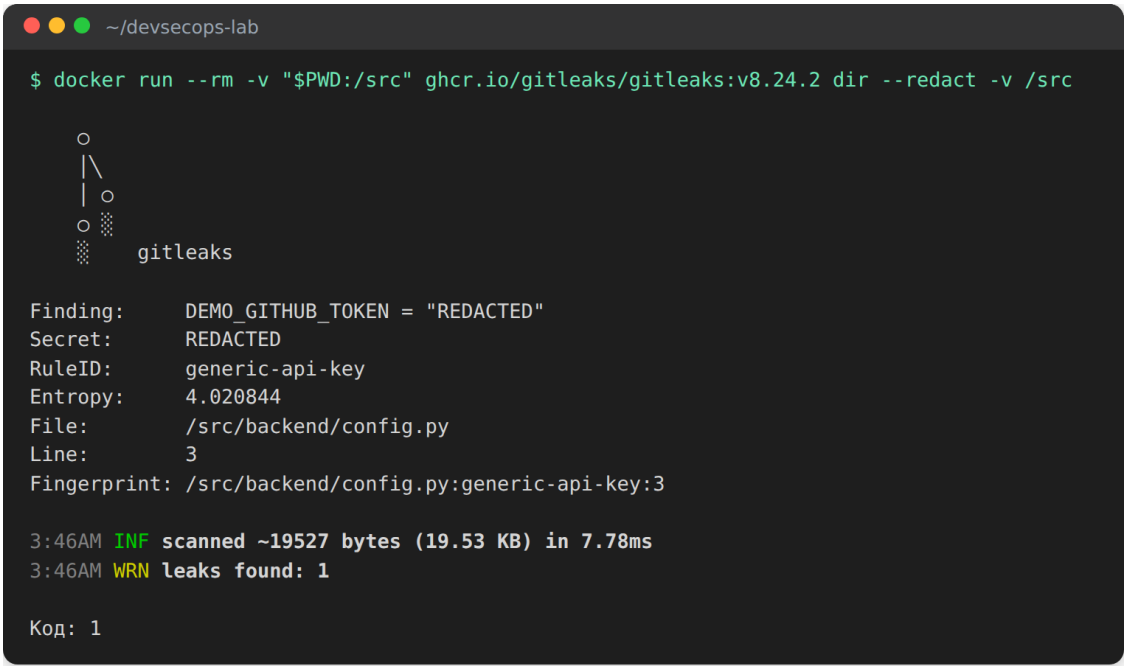
Часть команды	Смысл
`docker run --rm`	запустить контейнер и удалить его после работы
`-v "$PWD:/src"`	показать контейнеру текущий каталог под именем `/src`
`ghcr.io/gitleaks/gitleaks:v8.24.2`	образ сканера **с точной версией**
всё, что дальше	параметры самого сканера

Если `Cannot connect to the Docker daemon` — не запущен Docker Desktop.

Шаг A2. Найти секрет в файлах

```
docker run --rm -v "$PWD:/src" ghcr.io/gitleaks/gitleaks:v8.24.2 dir --redact -v
 /src
echo "Код: $?"
```

Параметр	Смысл
`dir`	проверять файлы в каталоге
`--redact`	скрыть значение секрета в выводе
`-v`	показать подробности каждой находки



Разберите находку по полям:

Поле	Что означает
`Finding`	строка, в которой найден секрет; значение скрыто

Поле	Что означает
`RuleID`	какое правило сработало
`Entropy`	энтропия строки; чем выше, тем «случайнее»
`File`, `Line`	где находится
`Fingerprint`	отпечаток находки

Что увидите: `leaks found: 1`, `Код: 1`. Код 1 — «найдены утечки»: это результат проверки, а не сбой.

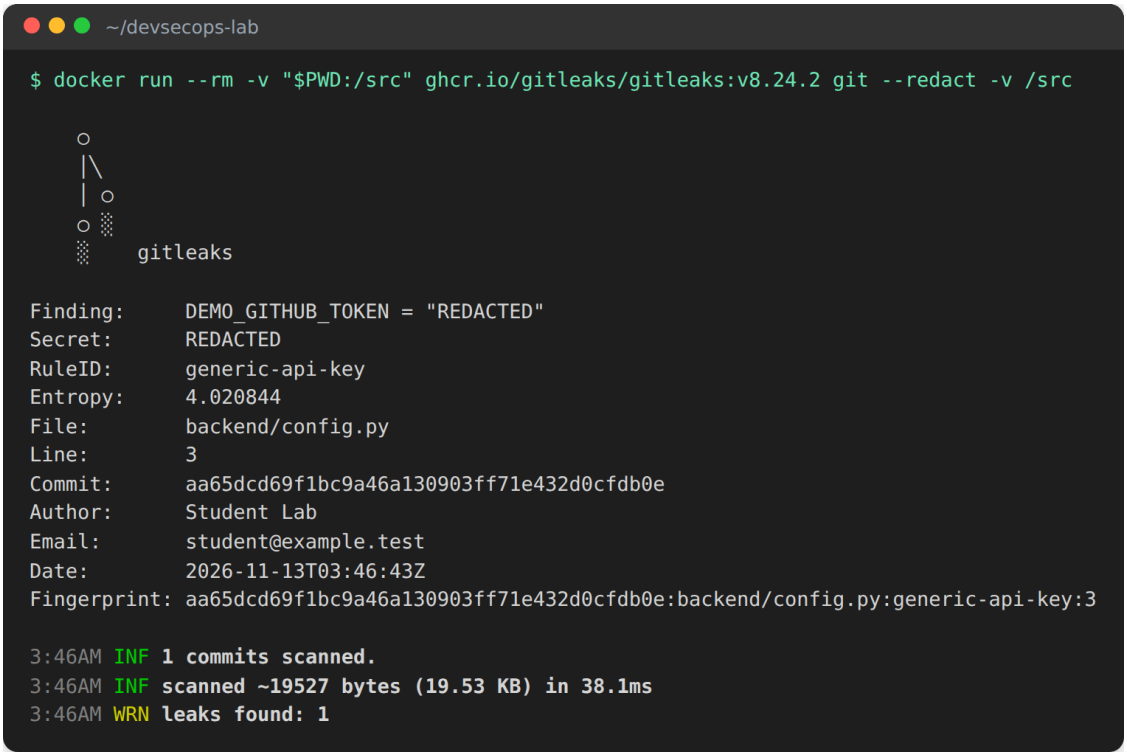
Обратите внимание: сработало правило `generic-api-key` (энтропия), а не правило для токенов GitHub. Учебный токен короче настоящего, точное правило его не узнало.

Зачем `--redact`? Вывод сканера попадает в журналы конвейера и в отчёты, которые читают многие. Без этого параметра секрет утёк бы ещё раз — уже через отчёт.

В отчёт: вывод команды. Значение секрета в отчёт не копируйте.

Шаг А3. Найти секрет в истории

```
docker run --rm -v "$PWD:/src" ghcr.io/gitleaks/gitleaks:v8.24.2 git --redact -v
 --report-format json --report-path /src/reports/gitleaks-before.json /src
```



Что увидите: та же находка с новыми полями — `Commit`, `Author`, `Date`. Отпечаток теперь начинается с идентификатора коммита.

Заполните карточку находки:

Поле	Значение

Поле	Значение
Правило	
Файл и строка	
Коммит (первые 7 символов)	
Автор и дата коммита	
Отпечаток	
К чему дал бы доступ настоящий токен GitHub	

В отчёт: карточка.

Шаг A4. Убрать секрет из кода

Готовое исправление переносит секрет в переменные окружения:

```
cp -r ../secure-development-labs/solutions/lab3-secrets/. .
git diff
```

Что увидите:

```
-DEMO_GITHUB_TOKEN = "ghp_…"
-FLASK_SECRET_KEY = "dev-only-change-me"
+DEMO_GITHUB_TOKEN = os.getenv("DEMO_GITHUB_TOKEN", "")
+FLASK_SECRET_KEY = os.getenv("FLASK_SECRET_KEY", "dev-only-change-me")
```

Значение токена теперь задаётся там, где приложение запускают. В коде осталось только имя переменной.

Проверьте и сохраните:

```
pytest -q
docker run --rm -v "$PWD:/src" ghcr.io/gitleaks/gitleaks:v8.24.2 dir --redact -v
 /src
git add -A
git commit -m "lab3: read secrets from environment"
```

Что увидите: `7 passed`, затем `no leaks found`.

Если `gitleaks dir` нашёл что-то в папке `reports` — это отчёты других сканеров, в которых встречаются длинные случайные строки. В коммиты эта папка не попадает; такие находки не учитывайте. Заодно это наглядный пример ложного срабатывания.

Шаг A5. Убедиться, что история помнит

```
docker run --rm -v "$PWD:/src" ghcr.io/gitleaks/gitleaks:v8.24.2 git --redact -v
 /src
git show HEAD~2:backend/config.py
```

`git show HEAD~2:файл` — показать файл таким, каким он был два коммита назад.

Что увидите: `leaks found: 1`, а вторая команда печатает старую версию файла вместе с токеном. Файл чист — секрет доступен.

Опишите порядок действий, будь токен настоящим. Заполните таблицу:

№	Действие	Кто выполняет	Где именно
1	Отозвать токен	владелец учётной записи GitHub	Settings → Developer settings → Personal access tokens
2	Выпустить новый и передать приложению		
3	Проверить журналы использования		
4	Убрать токен из кода		
5	Поставить автоматическую проверку		

В отчёт: вывод команд, таблица.

Шаг А6. Поставить блокировку коммитов

В проекте есть файл настроек `.pre-commit-config.yaml`:

```yaml
repos:
  - repo: https://github.com/gitleaks/gitleaks
    rev: v8.24.2
    hooks:
      - id: gitleaks
```

Поле	Смысл
`repo`	откуда взять проверку
`rev`	точная версия — та же, что вы запускали вручную
`id`	какую проверку подключить

```
pre-commit install
pre-commit run --all-files
```

Что увидите:

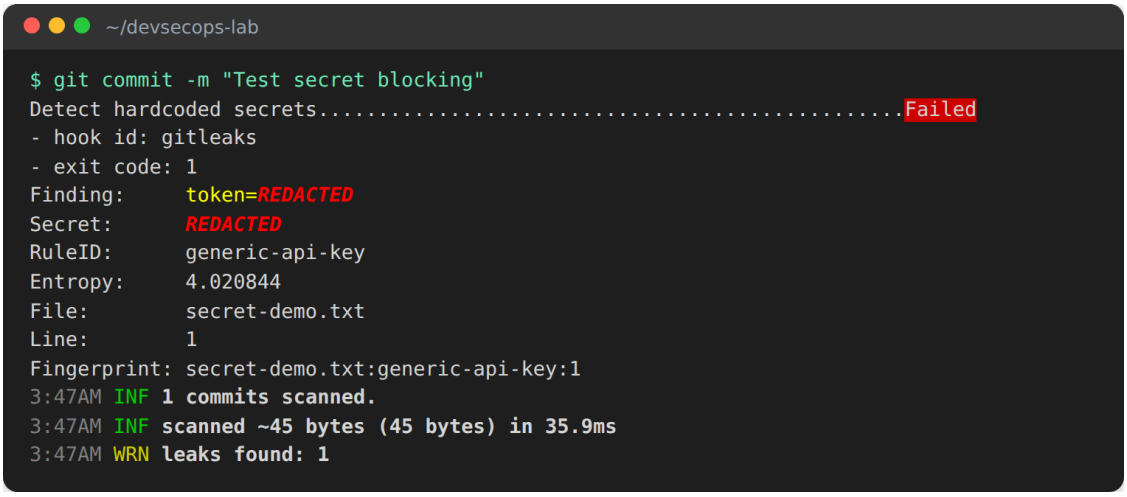
```
pre-commit installed at .git/hooks/pre-commit
Detect hardcoded secrets.....................................................Passed
```

Первый запуск занимает несколько минут: pre-commit скачивает и собирает Gitleaks.

Шаг А7. Проверить, что блокировка работает

Защиту, которую ни разу не видели в действии, нельзя считать работающей.
Попробуйте закоммитить файл с «секретом»:

```
printf 'token=ghp_8ae31cacf141669ddfb5da7e31c8a2f3a4\n' > secret-demo.txt
git add secret-demo.txt
git commit -m "Test secret blocking"
```



Что увидите: `Failed`, имя файла и строка. Коммит не создан — убедитесь и уберите файл:

```
git log --oneline
git restore --staged secret-demo.txt
rm secret-demo.txt
git status
```

В отчёт: вывод заблокированного коммита и `git log --oneline` (коммита `Test secret blocking` в нём нет).

Шаг А8. Оформить исключение для старой находки

Сейчас `gitleaks git` всегда завершается с ошибкой из-за первого коммита. В конвейере это означало бы вечно «красную» проверку. На такую быстро перестают смотреть — и пропускают новую, настоящую находку.

Находка разобрана: токен учебный, из кода удалён. Это основание для исключения. Получите отпечаток:

```
docker run --rm -v "$PWD:/src" ghcr.io/gitleaks/gitleaks:v8.24.2 git --redact -v
 /src | grep Fingerprint
```

Создайте в корне проекта текстовый файл `.gitleaksignore` из двух строк: комментарий с обоснованием и отпечаток **из вашего вывода** (у каждого студента он свой):

```
# Учебный токен из первого коммита: фиктивный, из кода удалён в ЛР3. <Фамилия>,
<дата>
162fb607f51d8fda36ca6ba640f692d4eb9bac0c:backend/config.py:generic-api-key:3
```

Проверьте и сохраните:

```
docker run --rm -v "$PWD:/src" ghcr.io/gitleaks/gitleaks:v8.24.2 git --redact -v
 /src
git add -A
git commit -m "lab3: document historical finding in .gitleaksignore"
```

Что увидите: `no leaks found`; при коммите хук показал `Passed`.

Если по-прежнему `leaks found: 1` — отпечаток скопирован неточно: он должен совпадать до символа, без пробелов по краям.

Правила исключений:

- исключается **одна находка по отпечатку**, а не правило и не файл целиком;
- рядом — обоснование, автор, дата;
- действующий настоящий секрет в исключения не вносят: его отзывают.

В отчёт: содержимое `.gitleaksignore` и ответ: чем это исключение отличается от попытки спрятать утечку?

Шаг А9. Чего Gitleaks не нашёл

Откройте `backend/config.py`. В нём осталось значение по умолчанию:

```
FLASK_SECRET_KEY = os.getenv("FLASK_SECRET_KEY", "dev-only-change-me")
```

Этим ключом приложение подписывает cookie сессии — по подписи оно узнаёт вошедшего пользователя. Gitleaks молчит: строка не похожа на случайную.

Ответьте в отчёте:

1. Что сможет сделать человек, который знает ключ подписи сессий?
2. Почему сканер пропустил этот ключ? Свяжите ответ с разделом 2.4.
3. К пропускам какого рода (раздел 2.4 ЛР2) относится этот случай?

Проверьте заодно Semgrep:

```
semgrep scan --config .semgrep.yml --metrics=off --error . ; echo "Код: $?"
```

Что увидите: `0 findings`, `Код: 0` — последняя находка из ЛР2 ушла.

В отчёт: три ответа.

Часть Б. Зависимости и SBOM

Команды Trivy длинные, поэтому заведите сокращение (действует до закрытия терминала; в новом окне выполните заново):

```
trivy() { docker run --rm -v "$PWD:/src" -v trivy-cache:/root/.cache/trivy -v /v
ar/run/docker.sock:/var/run/docker.sock aquasec/trivy:0.76.2 "$@"; }
```

Теперь `trivy …` запускает сканер версии 0.76.2 в контейнере. Каталог проекта виден ему как `/src`; `trivy-cache` хранит базу уязвимостей между запусками.

Шаг Б1. Просканировать зависимости

```
trivy --version
trivy fs --scanners vuln /src
```

Параметр	Смысл
`fs`	проверять файлы проекта
`--scanners vuln`	искать уязвимости в зависимостях

Первый запуск скачивает базу уязвимостей (1–2 минуты).

```
~/devsecops-lab
$ trivy fs --scanners vuln /src

Report Summary

┌──────────────────────────────────┬──────┬─────────────────┐
│              Target              │ Type │ Vulnerabilities │
├──────────────────────────────────┼──────┼─────────────────┤
│ legacy-component/requirements.txt │ pip  │       44        │
├──────────────────────────────────┼──────┼─────────────────┤
│ requirements.txt                 │ pip  │        1        │
└──────────────────────────────────┴──────┴─────────────────┘
Legend:
- '-': Not scanned
- '0': Clean (no security findings detected)
```

Что увидите: сводную таблицу, затем подробности по каждому файлу.

Файл	Уязвимостей в контрольном запуске
`legacy-component/requirements.txt`	44
`requirements.txt`	1

У вас чисел будет **больше**: с тех пор база пополнилась. Запишите свои числа и дату — в отчёте сравните с контрольными.

В отчёт: версия Trivy, дата запуска, сводная таблица.

Шаг Б2. Научиться читать таблицу уязвимостей

Покажите только критические:

```
trivy fs --scanners vuln --severity CRITICAL /src
```

```
~/devsecops-lab
$ trivy fs --scanners vuln --severity CRITICAL /src
legacy-component/requirements.txt (pip)
==========
Total: 3 (CRITICAL: 3)

┌─────────┬────────────────┬──────────┬────────┬───────────────────┬─────────────────┬──────────────────────────────────┐
│ Library │ Vulnerability  │ Severity │ Status │ Installed Version │  Fixed Version  │                Ti                │
├─────────┼────────────────┼──────────┼────────┼───────────────────┼─────────────────┼──────────────────────────────────┤
│ Django  │ CVE-2019-19844 │ CRITICAL │ fixed  │ 2.0.9             │ 1.11.27, 2.2.9, 3.0.1 │ Django: crafted email addre │
│         │                │          │        │                   │                 │ https://avd.aquasec.com/nvd      │
│         ├────────────────┤          │        │                   ├─────────────────┼──────────────────────────────────┤
│         │ CVE-2020-7471  │          │        │                   │ 1.11.28, 2.2.10, 3.0.3 │ django: potential SQL injec │
│         │                │          │        │                   │                 │ https://avd.aquasec.com/nvd      │
│         ├────────────────┤          │        │                   ├─────────────────┼──────────────────────────────────┤
│         │ CVE-2025-64459 │          │        │                   │ 5.2.8, 5.1.14, 4.2.26 │ django: Django SQL injectio │
│         │                │          │        │                   │                 │ https://avd.aquasec.com/nvd      │
└─────────┴────────────────┴──────────┴────────┴───────────────────┴─────────────────┴──────────────────────────────────┘
```

Колонка	Что означает
Library	библиотека
Vulnerability	идентификатор CVE
Severity	тяжесть
Status	`fixed` — исправление есть
Installed Version	версия в проекте
Fixed Version	версии, в которых исправлено; несколько значений — для разных веток библиотеки
Title	краткое описание и ссылка на подробности

Теперь покажите HIGH и CRITICAL вместе:

```
trivy fs --scanners vuln --severity HIGH,CRITICAL /src
```

Выберите по одной уязвимости для Django, requests, urllib3 и Jinja2. Откройте ссылку из колонки Title и заполните таблицу:

Библиотека	Версия	CVE	Тяжесть	Исправлено в	Суть одной фразой
requests	2.19.0	CVE-2018-18074	HIGH	2.20.0	при перенаправлении пароль уходит по незашифрованному каналу
Django					
urllib3					
Jinja2					

Сохраните полный отчёт в файл:

```
trivy fs --scanners vuln --format json --output /src/reports/trivy-fs-before.jso
n /src
```

В отчёт: таблица.

Шаг Б3. Сформировать SBOM

```
trivy fs --format cyclonedx --output /src/reports/sbom-before.cdx.json /src
```

Откройте `reports/sbom-before.cdx.json` в редакторе. Найдите:

- `bomFormat` и `specVersion` — формат и его версия;
- `components` — список компонентов; у каждого есть `type`, `name`, `version`.

Что увидите: CycloneDX, 9 компонентов: 7 библиотек (`library`) и 2 файла со списками зависимостей (`application`).

Вопрос: приложение использует библиотеку Werkzeug (проверьте: `pip list | grep -i werkzeug`), но в SBOM её нет. Почему? Подсказка: раздел 2.6. Вернётесь к этому в ЛР4.

В отчёт: формат, число компонентов, ответ.

Шаг Б4. Разобраться, что из найденного опасно

44 уязвимости в `legacy-component` — это много. Прежде чем бить тревогу, проверьте три факта.

Факт 1. Использует ли код эти библиотеки?

```
grep -rnE "^(import|from) +(django|requests|urllib3)" backend/
```

Что увидите: пусто — ни одного подключения.

Факт 2. Попадают ли они в образ? Откройте `Dockerfile`: какой файл передаётся команде `pip install`? Откройте `.dockerignore`: есть ли в нём `legacy-component`?

Факт 3. Есть ли они в собранном образе?

```
docker build -t devsecops-lab:local .
docker run --rm devsecops-lab:local pip list
```

Что увидите: Flask, Werkzeug, Jinja2 версии 3, psycopg, gunicorn. Django, requests и urllib3 в списке нет.

Запишите вывод по образцу:

> Компонент `legacy-component` есть в репозитории, но в работающее приложение не входит: (три доказательства). Риск для работающего сервиса — … . Риск для проекта — … . Решение: … .

Подсказки про риск для проекта: что будет, если кто-то этот файл установит? Как постоянный «красный» отчёт влияет на внимание к новым находкам?

В отчёт: три проверки и вывод.

Шаг Б5. Включить порог

Пока сканер только показывал таблицы. Чтобы он мог остановить конвейер, нужен порог:

```
trivy fs --scanners vuln --severity HIGH,CRITICAL --exit-code 1 /src
echo "Код: $?"
```

`--exit-code 1` — вернуть код 1, если найдены уязвимости указанной тяжести.

Что увидите: `Код: 1`. Без `--exit-code` Trivy всегда возвращает 0 — как Semgrep без `--error`.

Шаг Б6. Применить исправление и пройти порог

Два решения для двух ситуаций.

Ненужный компонент — удалить. Лучший способ избавиться от уязвимостей в неиспользуемом коде — убрать код:

```
git rm -r legacy-component
```

Нужную библиотеку — обновить:

```
cp -r ../secure-development-labs/solutions/lab3-deps/. .
git diff requirements.txt
```

Что увидите: `Flask==3.0.3` заменён на `Flask==3.1.3` — версию, в которой уязвимость исправлена.

После обновления библиотеки обязательно прогоните тесты: новая версия могла изменить поведение.

```
python -m pip install -r requirements.txt
pytest -q
```

Что увидите: `7 passed`.

Повторите сканирование с порогом:

```
trivy fs --scanners vuln --severity HIGH,CRITICAL --exit-code 1 /src
echo "Код: $?"
```

Что увидите: `requirements.txt | pip | 0`, `Код: 0`.

Объясните в отчёте выбор порога: почему в конвейере блокируют HIGH и CRITICAL, а не все находки подряд? Что случится с командой разработки, если блокировать и LOW?

Если в `requirements.txt` всё-таки нашлась HIGH — значит, после выхода методички опубликовали новую уязвимость. Запишите CVE и Fixed Version в отчёт и сообщите преподавателю: в жизни это происходит именно так.

В отчёт: оба вывода с кодами (до и после), объяснение порога.

Шаг Б7. SBOM «после» и правила приёмки

```
trivy fs --format cyclonedx --output /src/reports/sbom-after.cdx.json /src
```

Что увидите: 4 компонента вместо 9.

Заполните таблицу правил — что делать при каждом сигнале сканера. Она пригодится в ЛР5.

Сигнал	Блокировать изменение?	Действие
HIGH/CRITICAL в зависимостях, исправление есть	да	обновить, прогнать тесты
HIGH/CRITICAL в зависимостях, исправления нет		

Сигнал	Блокировать изменение?	Действие
MEDIUM и ниже		
Найден секрет		
Сканер не смог скачать базу уязвимостей		

Над последней строкой подумайте особенно: если сканер не отработал, проверка пройдена или не выполнена?

В отчёт: число компонентов, таблица.

Шаг Б8. Команды для конвейера

Вот как изученное будет выглядеть в конвейере в ЛР5:

```
# секреты
- uses: actions/checkout@v4
  with:
    fetch-depth: 0          # получить всю историю, а не один последний коммит
- name: Scan Git history
  run: gitleaks git --redact --no-banner .

# зависимости
- name: Scan dependencies
  run: docker run --rm -v "$PWD:/src" aquasec/trivy:0.76.2 fs --scanners vuln --
severity HIGH,CRITICAL --exit-code 1 /src
- name: Generate SBOM
  run: docker run --rm -v "$PWD:/src" aquasec/trivy:0.76.2 fs --format cyclonedx
 --output /src/reports/sbom.cdx.json /src
```

Те же команды, что вы выполняли руками. Версии сканеров указаны точно: сканер — такая же зависимость, как библиотека. Он получает доступ к коду и секретам сборки, и известны случаи подмены опубликованных версий популярных инструментов. `latest` означает «неизвестно что».

Шаг Б9. Сохранить результат

```
git add -A
git commit -m "lab3: update Flask, remove unused legacy manifest"
git log --oneline
```

Что увидите: хук показал `Passed`, в истории пять коммитов.

4. Требования к отчёту

Часть А

1. Вывод `gitleaks dir` и `gitleaks git` до исправления.
2. Карточка находки.
3. Результаты после исправления; доказательство, что секрет остался в истории.

4. Таблица реагирования на утечку.
5. Вывод заблокированного коммита.
6. Файл `.gitleaksignore` и ответ о его отличии от сокрытия.
7. Три ответа о ключе сессии.

Часть Б

8. Версия Trivy, дата, сводная таблица.
9. Таблица четырёх уязвимостей.
10. SBOM до и после, ответ про Werkzeug.
11. Разбор `legacy-component`: три проверки и вывод.
12. Сканирование с порогом до и после, объяснение порога.
13. Таблица правил приёмки.
14. Ответы на контрольные вопросы.

5. Контрольные вопросы

1. Почему удаление токена новым коммитом не устраняет утечку?
2. Чем режим `dir` отличается от режима `git`?
3. Зачем нужен `--redact`? Где ещё может «утечь» секрет, кроме кода?
4. Почему локального хука недостаточно и ту же проверку повторяют в CI?
5. Когда находку можно внести в `.gitleaksignore`, а когда нельзя?
6. Чем SCA отличается от SAST?
7. Почему тяжесть HIGH не означает автоматически высокий риск для проекта?
8. Зачем нужен SBOM, если есть сканер?
9. Почему число находок в неизменном проекте растёт со временем?
10. Что делает параметр `--exit-code 1` и почему версию сканера указывают точно?

6. Критерии оценивания

Критерий	Баллы
Часть А. Сканирование файлов и истории, карточка находки	15
Реагирование на утечку, проверка исправления	10
pre-commit: установка и доказательство блокировки	15
Обоснованное исключение, пропуск сканера	10
Часть Б. Сканирование зависимостей, таблица уязвимостей	15
SBOM	5
Разбор применимости (`legacy-component`)	15
Порог, правила приёмки	10
Контрольные вопросы	5
Итого	**100**

7. Источники

- ГОСТ Р 56939-2024. Разработка безопасного программного обеспечения. Общие требования.
- Gitleaks: https://github.com/gitleaks/gitleaks
- pre-commit: https://pre-commit.com/
- Trivy: https://trivy.dev/
- CycloneDX: https://cyclonedx.org/
- OWASP Secrets Management Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html
- База уязвимостей NVD: https://nvd.nist.gov/